



第五章 进程间通信





Review



- 进程的同步与互斥

- 临界资源 -->间接制约 -->临界区 -->互斥
- 进程间合作 -->直接制约 -->同步
- 互斥的方法：软件方法、硬件方法
- 信号量：
 - 整型信号量、记录型信号量、AND型信号量集、一般信号量集
- 使用信号量解决经典同步问题
 - 生产者-消费者问题、读者写者问题、哲学家就餐问题
- 管程：将同步互斥封装在一个模块内部



第五章 进程间通信IPC



Bridge in: 存在问题和需求

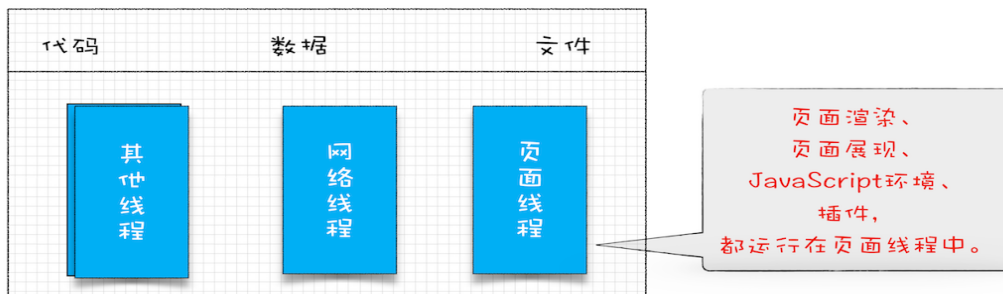
- **问题:** 一个进程对某个**信号量**的操作可使其它进程获得一些信息, 解决临界区互斥和进程间同步问题。但信号量的**信息量有限, 通信效率低**。
- **另一种需求:** 协作进程间需要大量的**信息交换**。各进程在执行过程中**合作完成**一项共同的任务, 需要协调步伐, 交流信息。协作的理由:
 - ✓ **信息共享(Information Sharing):** 多用户对同样信息感兴趣
 - ✓ **计算加速(Computation Speedup):** 多个子任务并行计算
 - ✓ **模块化(Modularity):** 可按模块化方式构建系统, 避免重复造轮子
 - ✓ **增强模块间隔离, 提供更强的安全保障**
 - ✓ **提高应用的容错能力。**
 - ✓ **方便(Convenience):** 单个用户也可能同时执行许多任务



第五章 进程间通信IPC

• 一个IPC例子：Chrome浏览器

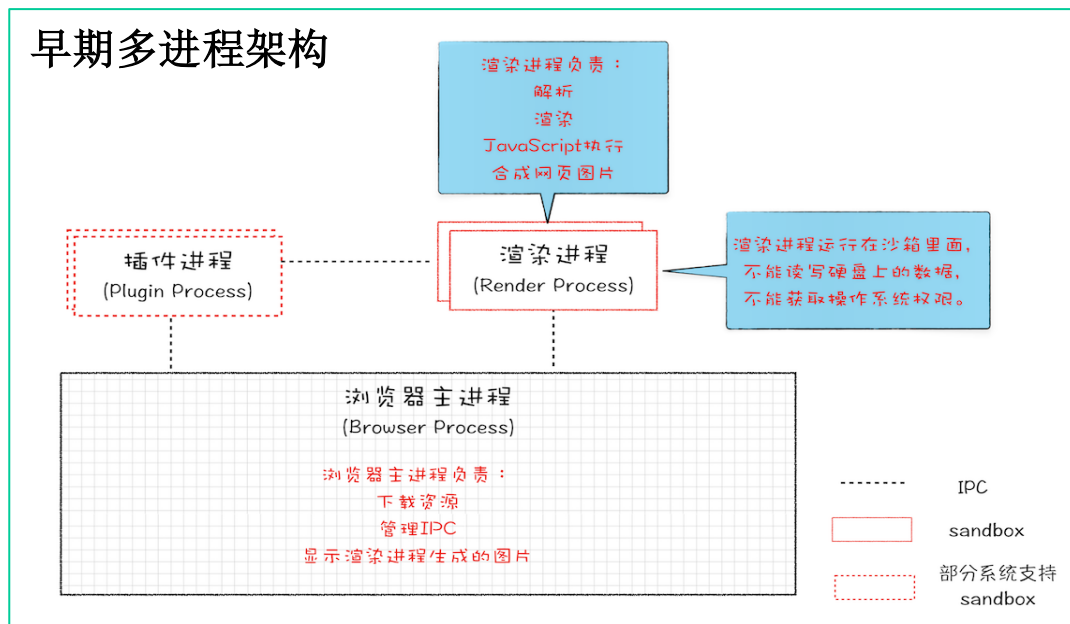
单进程浏览器



单进程浏览器的架构。即指浏览器的所有功能模块都是运行在同一个进程里，这些模块包含了网络、插件、JavaScript运行环境、渲染引擎和页面等。在2007年之前，市面上浏览器都是单进程的。

特点：不稳定、不流畅、不安全

早期多进程架构



这是2008年Chrome发布时的进程架构。由图可看出，Chrome的页面是运行在单独的渲染进程中，同时页面里的插件也是运行在单独的插件进程中，而进程之间是通过**IPC机制**进行通信（如图中虚线部分）





第五章 进程间通信



上一章讨论进程间的同步与互斥，利用信号量机制可以实现进程的并发控制，但是交换的信息通常是少量的控制信息，无法进行大量的数据交换。本章将讨论进程间的通信。

1. 概述
2. 进程的描述
3. 进程控制
4. 线程
5. 进程互斥和同步
- 6. 进程间通信 (Inter-Process Communication)**
- 7. 死锁问题**
- 8. 进程其他方面的举例**



第五章 进程间通信IPC



内容提要:

- ① 进程间通信的类型
- ② 共享存储区(shared memory)
- ③ 消息系统
- ④ 管道(pipe)
- ⑤ 套接字(socket)

目标:

- ① 了解进程间通信的几种方式;
- ② 能够清楚解释说明两种以上的进程间通信机制的原理
- ③ 在一个具体的OS下, 能够利用IPC有关的系统调用编写程序, 实现两个进程的批量信息交换。



5.1 进程间通信的类型



- 包括两种：低级通信和高级通信
- 低级通信：只能传递状态和整数值（控制信息），包括进程互斥和同步所采用的信号量和管程机制。
 - 优点：速度快。
 - 缺点：
 - 传送信息量小，效率低：每次通信传递的信息量固定，若传递较多信息则需要多次通信；
 - 编程复杂：用户直接实现通信的细节，编程复杂，容易出错。
- 高级通信：能够传送任意数量的数据，包括：共享存储区、管道、消息、套接字等。



5.1 进程间通信的类型



5.1.1 高级通信方式

(1) 共享存储器系统 (Shared-memory system)

相互通信的进程共享某些数据结构或共享存储区。一组进程向该公共区中写，另一组进程从公共区中读，因此又分为：

- 基于共享数据结构的通信方式
- 基于共享存储区的通信方式



5.1 进程间通信的类型

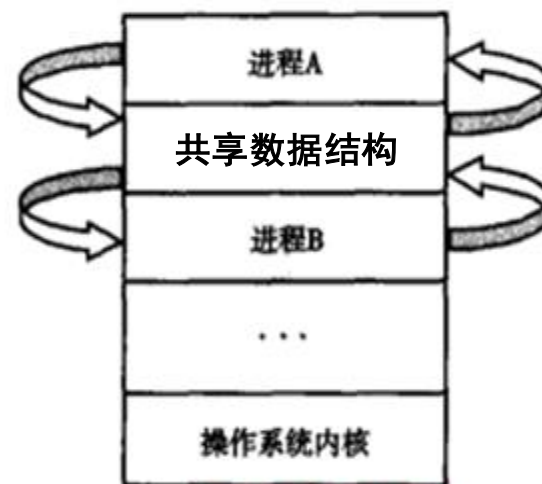


5.1.1 高级通信方式

(1) 共享存储器系统 (Shared-memory system)

● 基于共享数据结构的通信方式

- ✓ 进程之间能够通过**某种类型的数据结构**（如有界缓冲区）交换信息，如**生产者和消费者问题**。
- ✓ OS只负责提供共享存储区，而共享数据结构和对进程间的**同步处理都是程序员的事**。
- ✓ **特点：**通信效率低，只适合于传递少量信息。





5.1 进程间通信的类型



5.1.1 高级通信方式

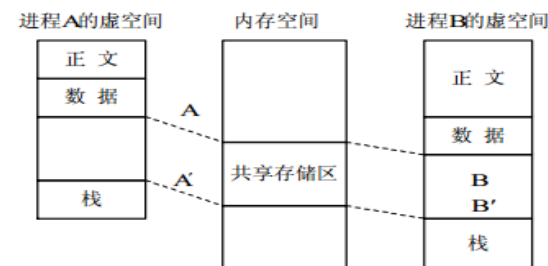
(1) 共享存储器系统 (Shared-memory system)

● 基于共享存储区的通信方式【后面详讲】

- 系统在存储器中划出一块**共享的存储区**，各进程间可通过对共享存储区中的数据进行读或写来实现通信；
- 进程在通信之前应向系统**申请**共享存储区中的一个分区，并指定该分区的**关键字**，若系统已经给进程分配了这样的分区，则将该分区的描述符返回给申请者；
- 接着，申请者把获得的共享存储区**连接到本进程上**，此后就像读写普通存储器一样；
- 主要用于UNIX System V中，Linux也支持。

✓ 特点：

- ✓ **高效性**：进程可直接访问共享内存区域，而不需要通过内核进行数据复制；
- ✓ **无需中介**：允许进程之间直接交换数据。
- ✓ **同步问题**：需要用户程序采取同步机制确保数据一致性





5.1 进程间通信的类型



5.1.1 高级通信方式

(2) 消息系统 (Message System)

- 进程间的信息交换以消息或报文为单位;
- 程序员利用系统提供的一组通信命令 (原语) 实现通信;
- OS隐藏了通信的实现细节, 简化了编程的复杂性。
- 分为两种:
 - 直接通信方式
 - 间接通信方式



5.1 进程间通信的类型



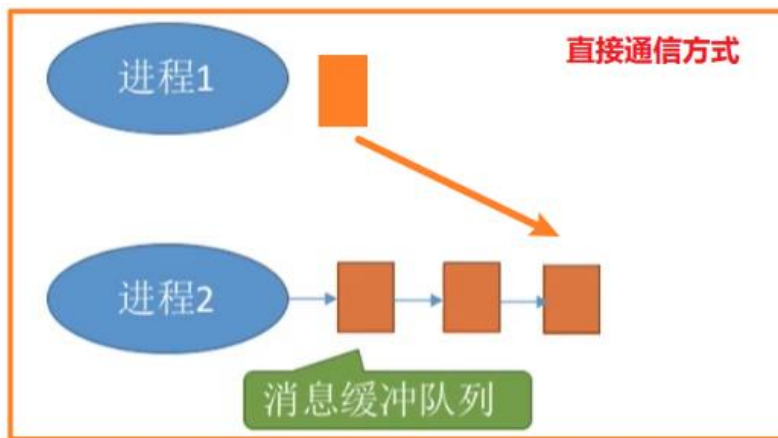
5.1.1 高级通信方式

(2) 消息系统 (Message System)

● 直接通信方式【后面详讲】

发送进程发消息时要指定接收进程的名字，反过来，接收时要指明发送进程的名字。系统提供两条原语：

- **Send (receiver,message)**
- **Receive (sender,message)**





5.1 进程间通信的类型

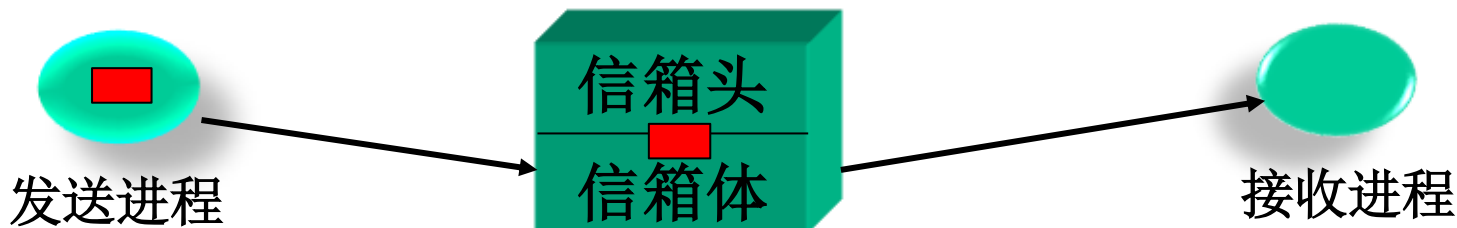


5.1.1 高级通信方式

(2) 消息系统 (Message System)

● 间接通信方式

- ✓ 收发双方进程通过某种**中间实体**，作为通信进程间的媒介，即**信箱 (MailBox)**；
- ✓ 发送进程发消息时不指定接收进程的名字，而是指定一个中间媒介；
- ✓ 通常收方和发方的数目可以是任意的。这种方式广泛应用于**多机系统和计算机网络**中。





5.1 进程间通信的类型



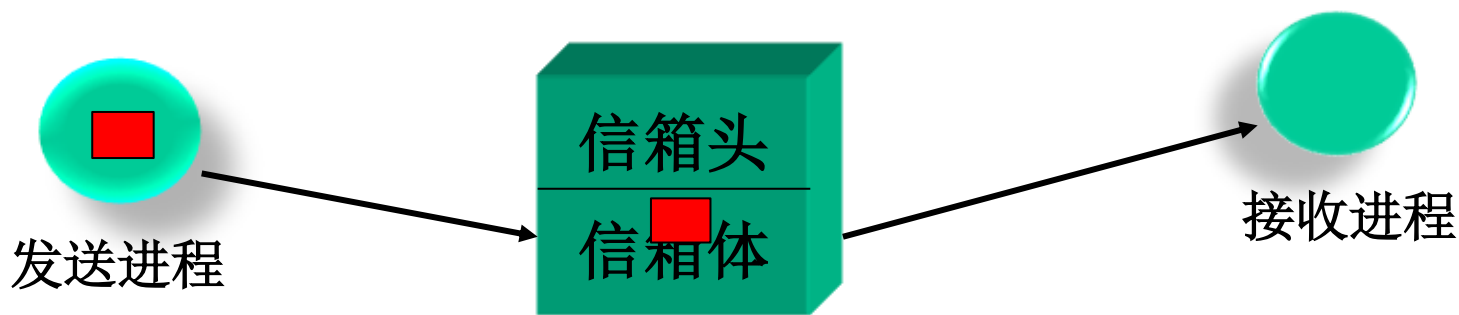
5.1.1 高级通信方式

(2) 消息系统 (Message System)

● 间接通信方式

➤ 发送原语: `send(MB, Message)`

➤ 接收原语: `receive(MB, Message)`





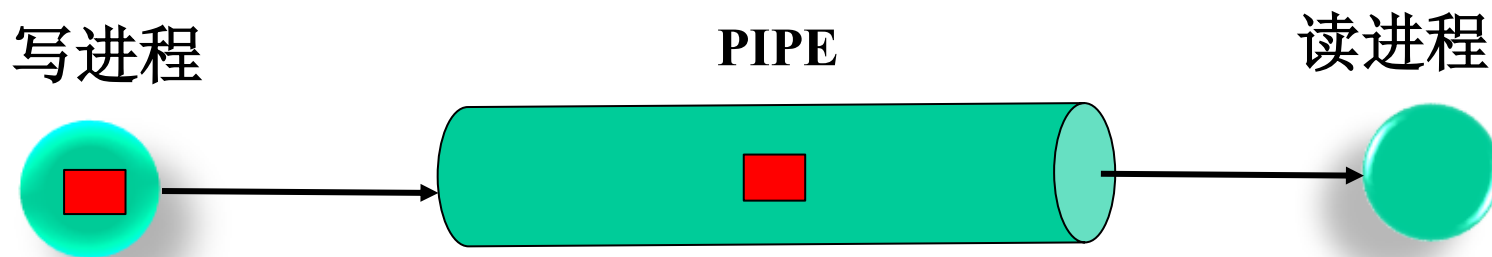
5.1 进程间通信的类型



5.1.1 高级通信方式

(3) 利用共享文件的通信方式

基于文件系统形成的一种通信方式，即利用共享文件实现进程间通信。如UNIX中的管道（PIPE）。





5.1 进程间通信的类型



5.1.1 高级通信方式

(3) 利用共享文件的通信方式

● 管道机制必须提供三方面的协调努力：

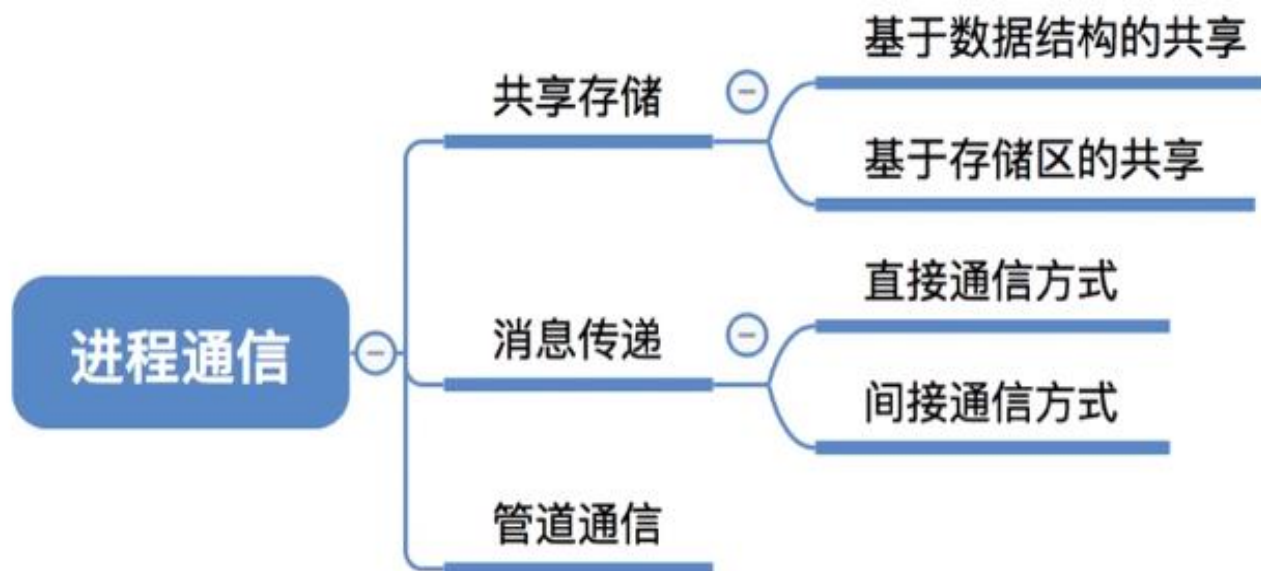
- **互斥**：当一个进程正在对PIPE进行读写时，另一个必须等待（一次只有一个进程可以访问）
- **同步**：当写进程把一定量的数据（如4KB）写入PIPE后（创建后，大小是固定字节的），便睡眠等待，直到读进程取走数据后再唤醒它；当读进程读一个空PIPE时，也应睡眠等待，直到写进程将消息写入管道为止，才将它唤醒
- **对方是否存在**：只有已确定对方存在时，方能进行通信



5.1 进程间通信的类型



5.1.1 高级通信方式--小结





5.2 消息系统 (Message System)



5.2.1 消息缓冲--原理

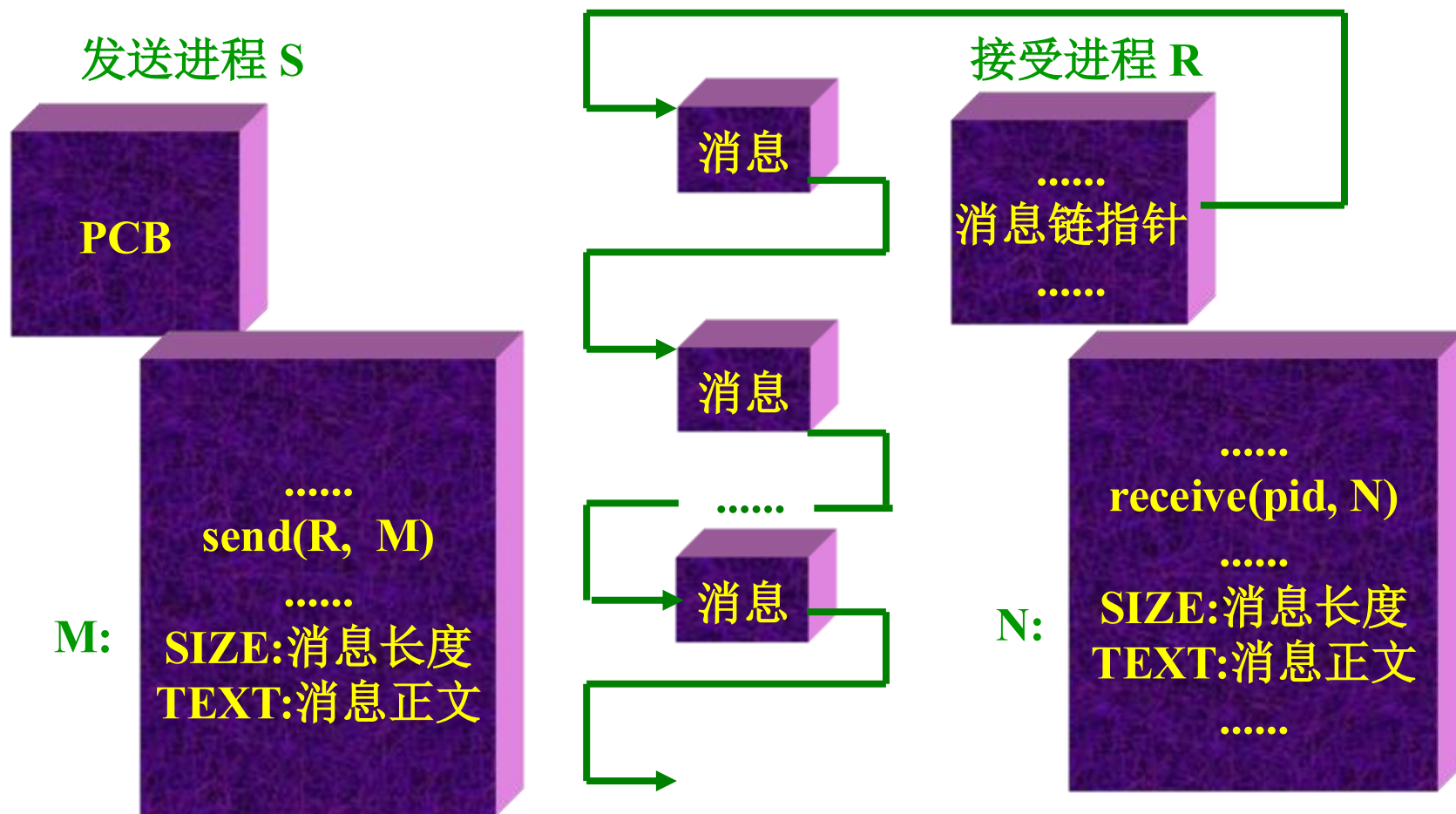
P. B. Hansen, 1973年在设计RC4000时提出的

- 系统在OS空间设置**一组缓冲区**，其中每个缓冲区可以存放一个消息；
- **发送过程**：当发送进程需要发送**消息**时，执行send系统调用，OS为发送进程**分配**一个空缓冲区，并将所发送的消息从发送进程**复制**到缓冲区中，然后将该载有消息的缓冲区**连接**到接收者进程的消息链链尾，完成发送；
- **接收消息**：在以后某个时刻，当接收进程执行到**receive**接收原语时，由OS将载有消息的缓冲区从消息链中**取出**，并把消息内容**复制**到接收进程空间，之后**收回**缓冲区，如此就完成了消息的接收。

5.2 消息系统 (Message System)



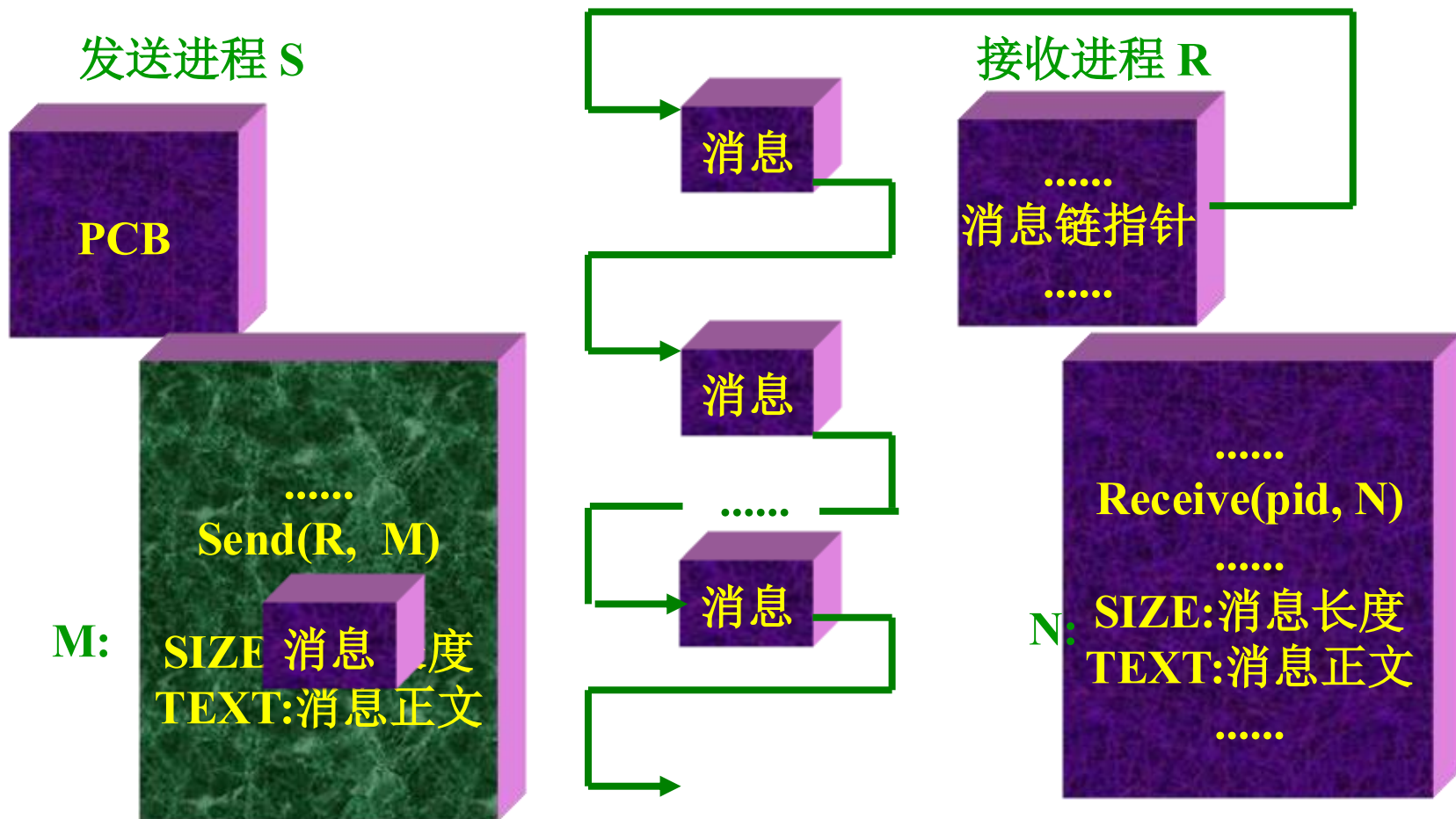
5.2.1 消息缓冲--通信模型





5.2 消息缓冲

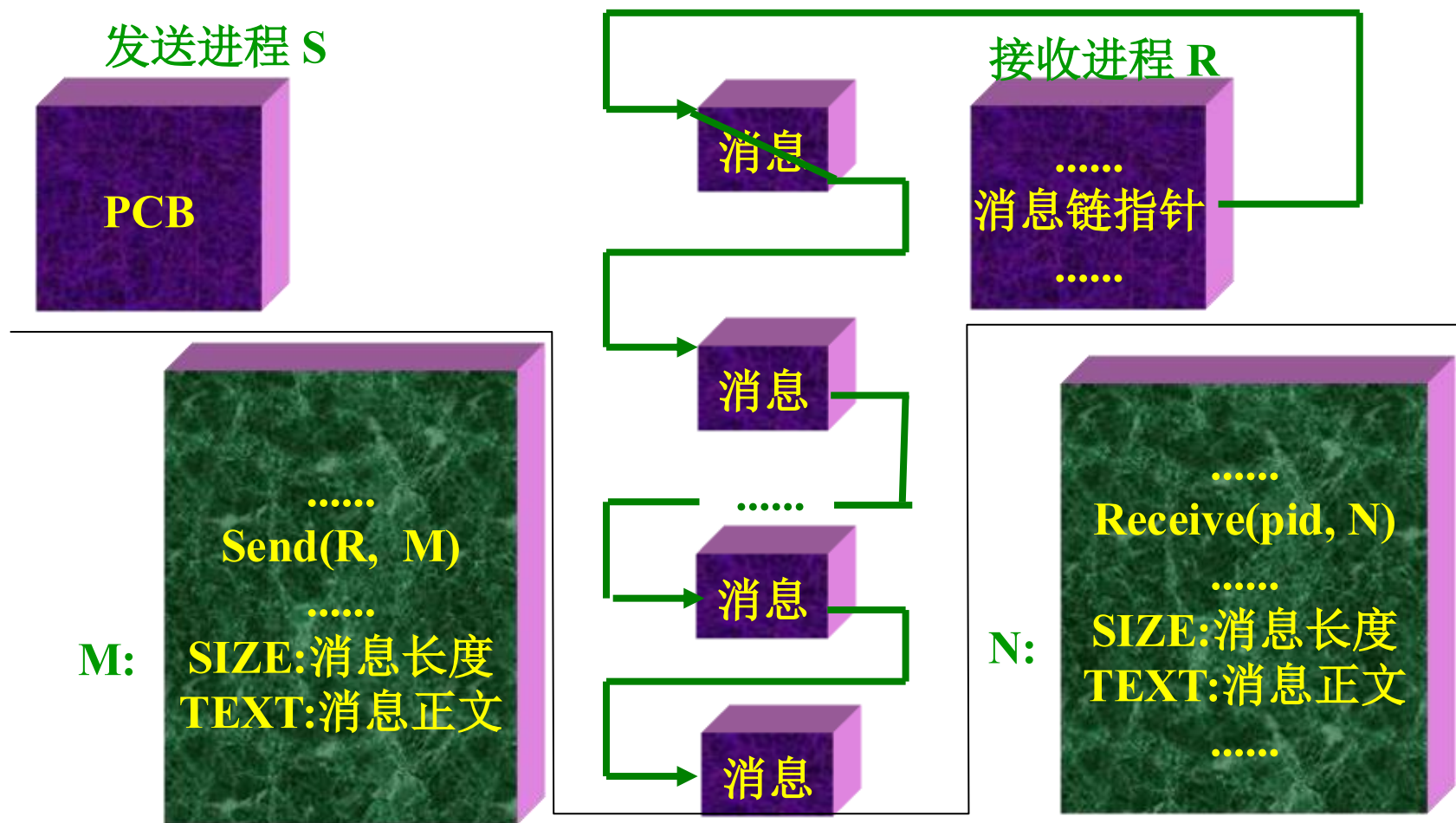
5.2.1 消息缓冲--通信模型





5.2 消息缓冲

5.2.1 消息缓冲--通信模型





5.2 消息缓冲



5.2.1 消息缓冲--消息缓冲区结构

```
typedef struct message_buffer {  
    int size;    //消息长度  
    char *text;  //消息正文  
    int  sendername; //发送者名字  
    ptrQueue next; //消息队列指针，用于链接消息  
}
```



5.2 消息缓冲



5.2.1 消息缓冲--对PCB的扩充

问题：基于上述的消息缓冲通信原理，这其中存在哪些并发控制问题？如何实现？

为实现消息缓冲通信，在PCB中增加：

- **Mutex：**消息队列互斥操作信号量；
- **Sm：**表示接收消息记数和同步信号量，表示消息队列中的消息数目；
- **Pm：**消息队列首指针



5.2 消息缓冲



5.2.1 消息缓冲--发送和接收原语

1. **Procedue send(receiver, a)**
 2. **Begin**
 3. **getbuf(a.size, i);** //申请缓冲区i,
 4. **i.sender := a.sender;** // 将发送区中的信息
 5. **i.size := a.size;** // 复制到消息缓冲区i中
 6. **i.text := a.text; i.next := 0;**
 7. **getid(PCBSet, receiver, j);** //获得接收进程的**内部标识符**
 8. **/*把缓冲区i挂入到receiver的消息队列中, 怎么做? */**
 9. **P(j.mutex);**
 10. **insert(j.Pm, i);** //将消息插入到接收者的消息队列
 11. **V(j.mutex);**
 12. **V(j.Sm);** //
 13. **End;**
- 问题1:** 这个i在哪里? 内核or用户空间?
- 问题2:** 为什么需要**V(j.Sm)**? 不加可以吗?

!! OS设计需要认真细致, 小心谨慎!



5.2 消息缓冲



5.2.1 消息缓冲--发送和接收原语

1. Procedure receive(b)
2. Begin
3. $j := \text{internal name};$ // 为接收者进程的內部标识
4. **P(j.Sm);**
5. P(j.mutex);
6. remove(j.Pm, i); // 将消息队列中的第一个消息移出
7. V(j.mutex);
8. b.sender := i.sender; //将消息缓冲区 i 中的信息
9. b.size := i.size; //复制到接收区b
10. b.text := i.text;
11. **putbuf(i);** // 归还空的缓冲区i
12. End;



5.2 消息缓冲



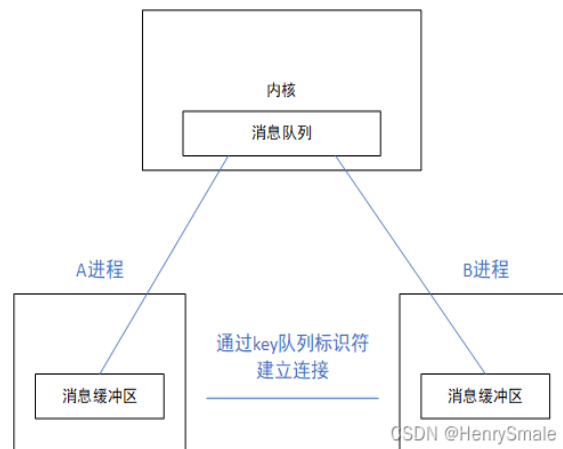
5.2.2 UNIX/Linux消息队列

● 消息队列(message queue):

- 每个message不定长,
- 由类型(type)和正文(text)组成

● UNIX消息队列API:

- **msgget**依据用户给出的整数值key, 创建新消息队列或打开现有消息队列, 返回一个消息队列ID;
 - `int msgget(key_t key, int msgflg);`
- **msgsnd**发送消息;
 - `int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);`
- **msgrcv**接收消息, 可指定消息类型; 没有消息时, 返回-1;
 - `ssize_t msgrcv(int msqid, void *msgp, size_t msgsz, long msgtyp, int msgflg);`
- **msgctl**对消息队列进行控制, 如删除消息队列;





5.2 消息缓冲



5.2.2 UNIX/Linux消息队列

- 通过指定多种消息类型，可以在一个消息队列中建立多个**虚拟信道**
- 注意：
 - 消息队列**不随创建它的进程的终止而自动撤销**，必须用msgctl(msgqid, IPC_RMID, 0)。
 - 另外，msgget获得消息队列ID之后，fork创建子进程，在子进程中**能**继承该消息队列ID而不必再一次msgget。



5.2 消息缓冲



5.2.3 Windows NT邮件槽

- **邮件槽(mailslot):** 驻留在OS核心, 不定长数据块(报文), 不可靠传递
- **通常采用client-server模式:** 单向的, 从client发往server; server负责创建邮件槽, 它可从邮件槽中读消息; client可利用邮件槽的名字向它发送消息;
- **在建立邮件槽时有一定的限制:**
 - 作为client(发送方)打开邮件槽时, 可以是“\\range\\mailslot\\[path]name”, 这里range可以是.(local computer), computerName, domainName, *(primary domain)(打开其他机器上的邮件槽);
 - 作为server(接收方)只能在本机建立邮件槽, 名字可为“\\.\\mailslot\\[path]name”;



5.2 消息缓冲



5.2.3 Windows NT邮件槽

● 有关的API

- ✓ **CreateMailslot:** 服务器方创建邮件槽，返回其句柄；
- ✓ **GetMailslotInfo:** 服务器查询邮件槽的信息，如：消息长度、消息数目、读操作等待时限等；
- ✓ **SetMailslotInfo:** 服务器设置读操作等待时限；
- ✓ **ReadFile:** 服务器读邮件槽；
- ✓ **CreateFile:** 客户方打开邮件槽；
- ✓ **WriteFile:** 客户方发送消息；

● 在邮件槽的所有服务器句柄关闭后，邮件槽被关闭。这时，未读出的报文被丢弃，所有客户句柄都被关闭。



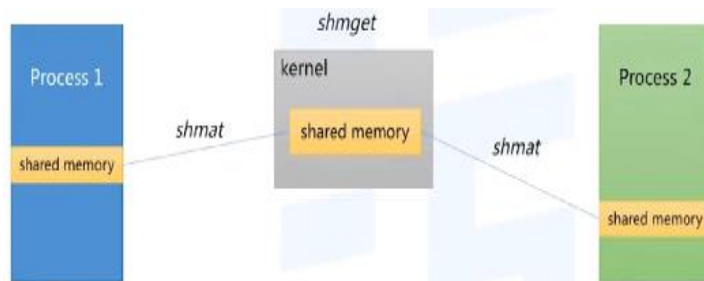
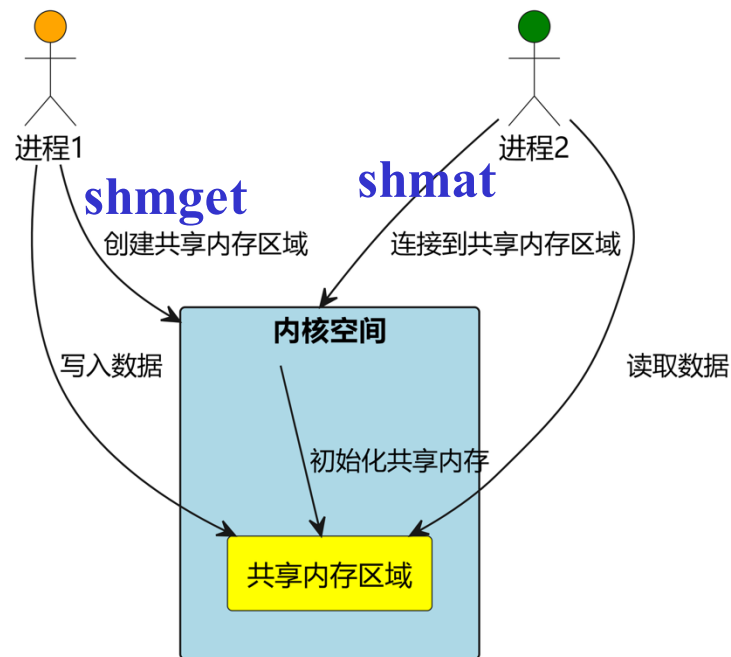
5.3 共享存储区(shared memory)



(1) UNIX的共享存储区

- **创建或打开共享存储区(shmget):** 依据用户给出的整数值key, 创建新区或打开现有区, 返回一个共享存储区ID。
- **连接共享存储区(shmat):** 把共享存储区连接到本进程的地址空间, 可以指定虚拟地址或由系统分配, 返回共享存储区首地址。父进程已连接的共享存储区可被其创建的**子进程继承**。
- **解除共享存储区连接(shmdt):** 拆除共享存储区与本进程地址空间的连接。
- **共享存储区控制(shmctl):** 对共享存储区进行控制。如: 删除共享存储区, 可调用shmctl(shmid, IPC_RMID, 0)。

使用共享内存传递数据的示例





利用共享存储区进行IPC示例



```
#include <sys/types.h>
#include <sys/shm.h>
#include <sys/ipc.h>
#define SHMKEY 75
int shmids, i;
int *addr;
void client( )
{
    int i;
    /*打开共享存储区*/
    shmids=shmget(SHMKEY,1024,0777);
    /*获得共享存储区首地址*/
    addr=shmat(shmids,0,0);
    for (i=9; i>=0; i--)
    {
        while (*addr !=-1 );
        printf("(client) sent\n");
        *addr=i;
    }
    exit(0);
}
```

```
void server()
{
    /*创建共享存储区*/
    shmid=shmget(SHMKEY,1024,0777|IPC_CREAT);
    addr=shmat(shmid,0,0); /*获取首地址*/
    do
    {
        *addr = -1;
        while (*addr == -1);
        printf("(server) received\n");
    } while (*addr);
    shmctl(shmid,IPC_RMID,0); /*撤消共享存储区, 归还资源*/
    exit(0);
}
```

```
main( )
{
    while ((i=fork( ))==1);
    if (!i) // Child process
        server();
    system("ipcs -m");
    while ((i=fork( ))==1);
    if (!i)
        client( );
    wait(0);
    wait(0);
}
```

ipcs 命令往标准输出
写入一些关于活动
进程间通信设施的
信息. **-m**共享内存段



5.3 共享存储区(shared memory)



(2) UNIX的文件映射

- **mmap**: 建立进程地址空间到文件或共享存储区对象的映射, 将文件偏移off开始的len字节映射到进程地址addr处:

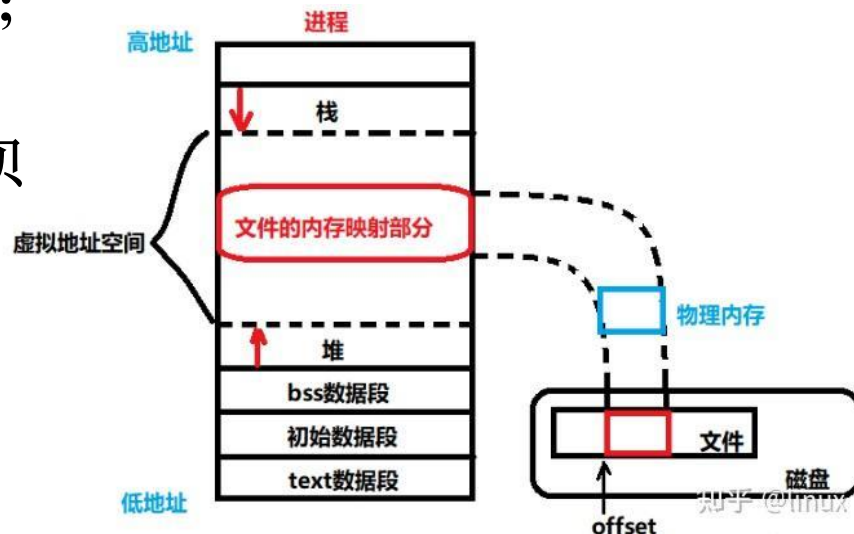
```
caddr_t mmap(caddr_t addr, size_t len, int prot,  
              int flags, int fildes, off_t off);
```

- **munmap**: 拆除映射

```
int munmap(void *addr, size_t len);
```

- **优点**:

- **提高效率**, 消除不必要的`数据拷贝`
- **降低复杂性**, 直接读写操作而不必管理缓冲区
- 可直接将内存的数据类型记录在文件上(如指针)





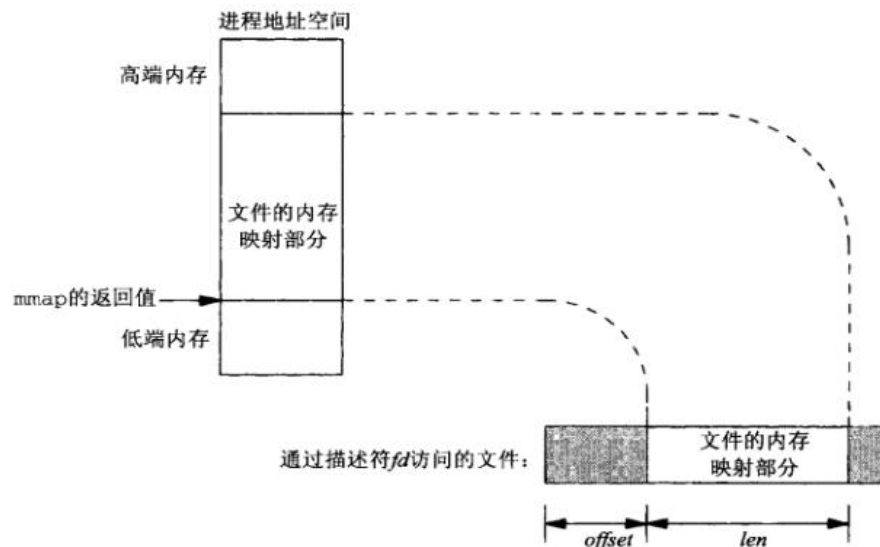
5.3 共享存储区(shared memory)



(2) UNIX的文件映射

```
#include <iostream>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <cstring>
using namespace std;

int main(int argc, char** argv){
    if(3!=argc){
        printf("Usage:%s filepath size\n",argv[0]);
        return 1;
    }
    int fd = open(argv[1],O_RDONLY);
    void* buff = mmap(NULL,stoi(argv[2]),PROT_READ,MAP_SHARED,fd,0);
    if(buff == MAP_FAILED){ // 测试文件是否0映射失败
        perror("mmap error");
        return 1;
    }
    cout << static_cast<char*>(buff) << endl; // 显示被映射的文件内容
    munmap(buff,stoi(argv[2])); // 将文件映射回去
    close(fd);
}
```



源自: https://blog.csdn.net/weixin_50941083/article/details/122565716



5.4 管道(pipe)



- **定义：**管道是一条在进程间以**字节流**方式传送信息的通信通道。
- 它由OS核心的缓冲区（通常几十KB）来实现，是**单向的**；常用于命令行所指定的**输入输出重定向和管道命令**。
（先进先出的，一个进程写，另一个进程读）
 - `$ ls /bin /usr/bin | sort | uniq | less`
 - 将ls的输出先排序，然后删除重复行，再分页显示：
- 在使用管道前要建立相应的管道，然后才可使用。





5.4 管道(pipe)



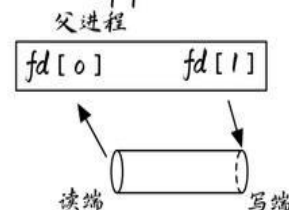
(1) UNIX管道

- 分两种：无名管道和命名管道
- 通过pipe系统调用创建无名管道，得到两个文件描述符，分别用于写和读。

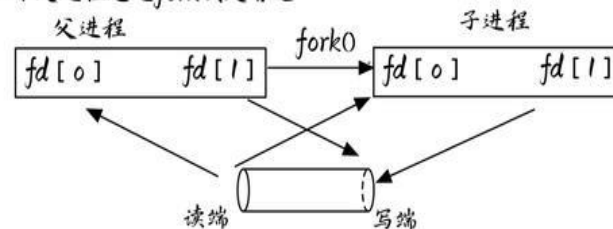
- `int pipe(int fildes[2]);`
- 文件描述符fildes[0]为读端，fildes[1]为写端；
- 通过系统调用write和read进行管道的写和读；
- 若进程间要进行双向通信，怎么办？
 - ✓ 通常需要两个管道；
- 只适用于父子进程之间或父进程安排的各个子进程之间（只有相关进程可以共享无名管道）；

匿名管道使用步骤

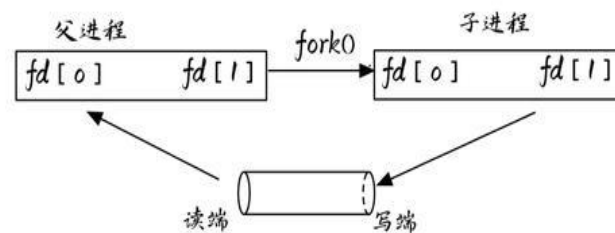
一、父进程通过pipe创建管道



二、父进程通过fork创建管道



三、父进程关闭写端，子进程关闭读端





5.4 管道(pipe)



(1) UNIX管道

● UNIX中的命名管道

- 可通过mknod()或mkfifo()系统调用建立:

int **mknod**(const char *path, mode_t mode, dev_t dev);

其中指定mode为**S_FIFO**

- 可以使用在系统中任意两个进程之间进行通信，且创建的管道文件存储在硬盘上，不会随着进程结束而消失。
- 不同于**无名管道**的是，它提供一个**路径与之关联**，以FIFO的文件形式存在于系统中。它在磁盘上有对应的**节点**，但没有数据块
- 只要进程对FIFO有适当的访问权，都可以通过**文件名**将其打开和进行读写，而不局限于父子进程，
- 参见：Linux/Unix进程间的通信之命名管道，https://blog.csdn.net/qq_44045338/article/details/104976246



5.4 管道(pipe)



```
#include <stdio.h>
main()
{
    int x, fd[2];
    char buf[30],s[30];
    pipe(fd);
    while((x=fork())!=-1);
    if(x==0) //子进程
    { sprintf(buf,"This is an example\n");
      write(fd[1],buf,30);
      exit(0);
    }
    else //父进程
    { wait(0);
      read(fd[0],s,30);
      printf("%s",s);
    }
}
```



5.4 管道(pipe)



(2) Windows NT管道

● 无名管道

类似于UNIX管道，CreatePipe可创建无名管道，得到两个读写句柄；利用ReadFile和WriteFile可进行无名管道的读写；

```
BOOL CreatePipe( PHANDLE hReadPipe,  
                // address of variable for read handle  
                PHANDLE hWritePipe,  
                // address of variable for write handle  
                LPSECURITY_ATTRIBUTES lpPipeAttributes,  
                // pointer to security attributes  
                DWORD nSize // number of bytes reserved for pipe  
                );
```



5.4 管道(pipe)



(2) Windows NT管道

命名管道

一个服务器端与一个客户进程间的通信通道；
可用于不同机器上进程通信

- ✓类型分为：字节流，消息流（报文）；
- ✓访问分为：单向读，单向写，双向；
- ✓还有关于操作阻塞(wait/nwait)的设置。相应有一个文件句柄



5.4 管道(pipe)



(2) Windows NT管道

命名管道

- 通常采用client-server模式，连接本机或网络中的两个进程
- 管道名字
 - 作为客户方（连接到一个命名管道实例的一方）时，可以是"`\\serverName\pipe\pipename`";
 - 作为服务器方(创建命名管道的一方)时，只能取serverName为`\\.\pipe\PipeName`，不能在其它机器上创建管道；



5.4 管道(pipe)



(2) Windows NT管道

● **命名管道服务器支持多客户：**为每个管道实例建立单独线程或进程。

- **CreateNamedPipe**在服务器端创建并返回一个命名管道句柄；
- **ConnectNamedPipe**在服务器端等待客户进程的请求；
- **CallNamedPipe**从管道客户进程建立与服务器的管道连接；
- **ReadFile**、**WriteFile**（用于阻塞方式）、**ReadFileEx**、**WriteFileEx**（用于非阻塞方式）用于命名管道的读写。



5.5 套接字(socket)



- 双向的，数据格式为字节流（一对一）或报文（多对一，一对多）；主要用于网络通信；
- 支持client-server模式和peer-to-peer模式，本机或网络中的两个或多个进程进行交互；提供TCP/IP协议支持
- UNIX套接字(基于TCP/IP):
 - send, sendto, recv, recvfrom;
- 在Windows NT中的规范称为“Winsock”(与协议独立，或支持多种协议):
 - WSASend, WSA Sendto, WSARecv, WSARecvfrom。



5.6 信号 (Signal)

(1) 信号是什么?

- 信号是Unix、类Unix以及其他POSIX兼容的操作系统中**进程间通讯的一种有限制**的方式。
- **软中断信号** (signal, 简称为信号), 它是一种**异步的通知机制**, 用来通知进程发生了异步事件。
- 当一个信号发送给一个进程, **OS中断进程**正常的控制流程, 此时, 任何非原子操作都将被中断。如果进程定义了信号的处理函数, 那么它将被执行, 否则就执行默认的处理函数。
- 进程之间可以互相通过系统调用**kill**发送软中断信号。**内核**也可以因为内部事件而给进程发送信号, 通知进程发生了某个事件
- 信号只是用来通知某进程发生了什么事, 并不**给该进程传递任何数据**。



5.6 信号 (Signal)

(2) 信号的类型

发出信号的原因很多，按发出信号的原因简单分类：

- (1) **与进程终止相关的信号**。当进程退出，或者子进程终止时，发出这类信号。
- (2) **与进程例外事件相关的信号**。如进程越界，或企图写一个只读的内存区域（如程序正文区），或执行一个特权指令及其他各种硬件错误。
- (3) **与在系统调用期间遇到不可恢复条件相关的信号**。如执行系统调用exec时，原有资源已经释放，而系统资源又已经耗尽。
- (4) **与执行系统调用时遇到非预测错误条件相关的信号**。如执行一个并不存在的系统调用。
- (5) **在用户态下的进程发出的信号**。如进程调用系统调用kill向其他进程发送信号。
- (6) **与终端交互相关的信号**。如用户关闭一个终端，或按下break键等情况。
- (7) **跟踪进程执行的信号**。



5.6 信号 (Signal)

(2) 信号的类型

Linux支持的信号列表：

信号	值	处理动作	发出信号的原因
SIGHUP	1	A	终端挂起或者控制进程终止
SIGINT	2	A	键盘中断（如break键被按下）,Ctrl+del
SIGQUIT	3	C	键盘的退出键被按下
SIGILL	4	C	非法指令
SIGABRT	6	C	由abort(3)发出的退出指令
SIGFPE	8	C	浮点异常
SIGKILL	9	A E F	Kill信号
SIGSEGV	11	C	无效的内存引用
SIGPIPE	13	A	管道破裂：写一个没有读端口的管道
SIGALRM	14	A	由alarm(2)发出的信号
SIGTERM	15	A	终止信号
SIGUSR1	30,10,16	A	用户自定义信号1
SIGUSR2	31,12,17	A	用户自定义信号2
SIGCHLD	20,17,18	B	子进程结束信号
SIGCONT	19,18,25		进程继续（曾被停止的进程）
SIGSTOP	17,19,23	D E F	终止进程
SIGTSTP	18,20,24	D	控制终端（tty）上按下停止键
SIGTTIN	21,21,26	D	后台进程企图从控制终端读
SIGTTOU	22,22,27	D	后台进程企图从控制终端写



5.6 信号 (Signal)



(3) 发送信号的方式

- **通过键盘发送信号**，只针对前台进程；
 - 常见的有 `ctrl + c` , `ctrl + z` , `ctrl + ;`;都是用来终止进程的。
- **进程异常**，产生信号
 - 当进程出现异常时，如被零除、非法地址访问，会产生信号，一般这种情况我们叫做程序崩溃。
- **通过调用系统调用接口函数**，向进程发送信号
 - `kill` 命令就是用的`kill`函数实现的，比如要杀死某个进程 `kill -9 PID`
- **满足某些软件条件**，也可以产生信号
 - 比如之前学到命名管道，如果读端已经关闭，写端还在写入，那么系统会发送信号 13 来终止进程。
 - 如**alarm函数**和SIGALRM信号。`alarm()`的作用是，设置一个时间，时间一到就会发送信号SIGALRM，默认行为是终止当前进程。



5.6 信号 (Signal)



(4) 信号的控制

➤ 几个概念:

- 实际执行信号的处理动作称为**信号递达**(Delivery)
- 信号从产生到递达之间的状态, 称为**信号未决**(Pending)。
- **信号阻塞**: 进程可以选择阻塞 (Block) 某个信号。被阻塞的信号产生时将保持在未决状态, 直到进程解除对此信号的阻塞, 才执行递达的动作。
- 阻塞和忽略是不同的, 只要信号被阻塞就不会递达, 而忽略是在递达之后可选的一种处理动作。



5.6 信号 (Signal)



(4) 信号的控制

➤ 信号的流程:

- 操作系统发来信号，进程保存信号，看这个信号是否被阻塞？
- 若是阻塞，则就先保存着；若不阻塞则就在合适的时机递达信号。
- 递达后有三种情况：
 - ✓ 执行信号的默认行为：
 - 显式地忽略信号 (Ignore)：即内核将会丢弃该信号，信号不会对目标进程产生任何影响。
 - 终止进程 (Terminate)：很多信号的默认处理是终止进程，即将进程杀死。
 - 生成核心转储文件并终止进程 (Core)：进程被杀死，并且产生核心转储文件。核心转储文件记录了进程死亡现场的信息。用户可以使用核心转储文件来调试，分析进程死亡的原因。
 - 停止进程 (Stop)：使进程暂停，将进程的状态设置成 TASK_STOPPED，一旦收到恢复执行的信号，进程还可以继续执行。
 - 恢复进程的执行 (Continue)：和停止进程相对应，某些信号可以使进程恢复执行。
 - ✓ 显式地忽略信号 (Ignore)：即内核将会丢弃该信号，信号不会对目标进程产生任何影响
 - ✓ 捕捉信号: 执行自定义的信号行为。当信号到达进程后，执行相应的信号处理函数



5.6 信号 (Signal)



(5) 信号的安装 (注册)

- **需求：**很多情况下，默认的信号处理函数，可能并不能满足实际的需要，这时需要修改信号的信号处理函数，而不执行默认的信号处理函数。
- **信号的安装：**为信号指定新的信号处理函数的动作，被称为~。
- glibc提供了signal函数和sigaction函数来完成信号的安装。signal出现得比较早，接口也比较简单，sigaction则提供了精确的控制

- `#include <signal.h>`
`void (*signal(int signo, void (*func)(int)))(int);`

```
int sigaction( int signo, const struct sigaction *restrict act, struct sigaction * restrict oldact);
```

```
struct sigaction {  
    void (*sa_handler)(int);  
    void (*sa_sigaction)(int, siginfo_t *, void *);  
    sigset_t sa_mask;  
    int sa_flags;  
    void (*sa_restorer)(void);  
};
```



5.6 信号 (Signal)



(6) 信号的表示与处理

- 信号的表示，每个信号有：

- 两个标志位：

- 1>block:该信号是否被屏蔽。

- 2>pending：是否收到信号。

- 一个handler: 函数指针，表示处理动作。

- 信号的处理

- Case1：信号未被屏蔽。没有收到信号，不会递达；收到信号，会递达，执行handler处理动作。

- Case2：信号被屏蔽。没有收到信号，不会递达；收到信号，也不会递达，当屏蔽解除时，信号递达，执行handler处理动作。若在解除屏蔽之前此信号产生多次，普通信号只递达一次，处理一次，实时信号会放进一个队列里，多次递达)



5.6 信号 (Signal)

(6) 信号的表示与处理

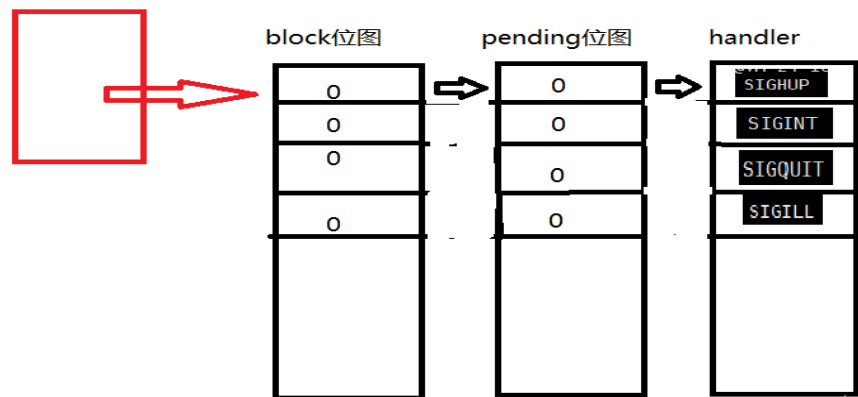
● 信号的实现机制

➤ 为实现信号机制，在PCB（或进程描述符）中有相关的字段：

- **sigset_t blocked**：是一个信号集，用于存储当前被阻塞的信号。当一个信号被阻塞时，即使它被发送到进程，也不会立即被处理，而是等待到阻塞被解除
- **sigset_t sigpending**：是一个信号集，用于存储当前挂起的信号。当一个信号被发送到进程，但尚未被处理时，它会被添加到此信号集中。
- **struct k_sigaction sigaction**：是一个结构体数组，每个元素对应一个信号。它包含了信号的处理方式（例如，是忽略、捕获、还是默认处理），以及处理该信号的函数指针（如果该信号被捕获）。

●

struct task_struct



CSDN @小小说



5.7 Linux的进程间通信



- 在linux下的多个进程间的通信机制叫做IPC，它是多个进程之间相互沟通的一种方法。
- 在linux下有多种进程间通信的方法：
 - ① 半双工管道
 - ② 命名管道
 - ③ 消息队列
 - ④ 共享内存
 - ⑤ 内存映射文件
 - ⑥ 套接字
 - ⑦ 信号
 - ⑧ 信号量
- 使用这些机制可以为linux下的网络服务器开发提供灵活而又坚固的框架。



5.7 Linux的进程间通信



Linux的匿名管道示例:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <string.h>
//匿名管道的创建与使用
```

```
int main() {
    int fd[2];
    char buf[1024]={" "};
    int pid;
    int states;
    if(pipe(fd)==-1) {
        printf("fail!\n");
    }
    pid=fork();
    if(pid<0) {
        printf("creat child fail!\n");
    }
}
```

(接右侧) 详见: Linux学习之进程通讯(IPC)—进程间通信, <https://blog.csdn.net/oneryjhhh/article/details/126534712>

(接左侧)

```
else if(pid>0) { //父进程
    printf("This is Father\n");
    sleep(3);
    close(fd[0]); //管道0负责读 1负责写
    write(fd[1], "Hello from father",
        strlen("Hello from father"));
    wait(&states);
}
else { //子进程
    printf("This is child\n");
    close(fd[1]);
    read(fd[0], buf, 1000);
    printf("read from father: %s\n", buf);
    exit(0);
}
return 0;
```

}



5.7 Linux的进程间通信

进程之间的通信IPC



CSDN @ONERYJHHH



进程间通信IPC--总结



高级、
低级

关于进程之间的通信

● 信号量、缓冲区

● 消息缓冲（缓冲池、信箱、...）

- 发送原语、接收原语

- 发送、接收进程的同步

- 缓冲媒介互斥

● 管道通信

- 无名管道（相关进程间）、命名管道（C/S）

- 管道的读写、进程同步、管道互斥（加锁等）

● 共享存储区

- 存储区的创建或打开、与进程的连接和拆除、对存储区的控制等

直接、
间接



小结



多道程序系统并发



程序失去原有的特点



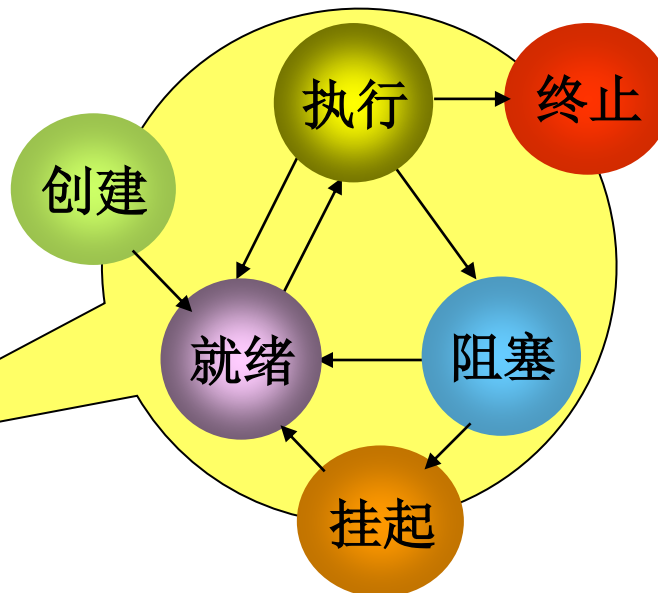
静态

程序 → 进程

动态

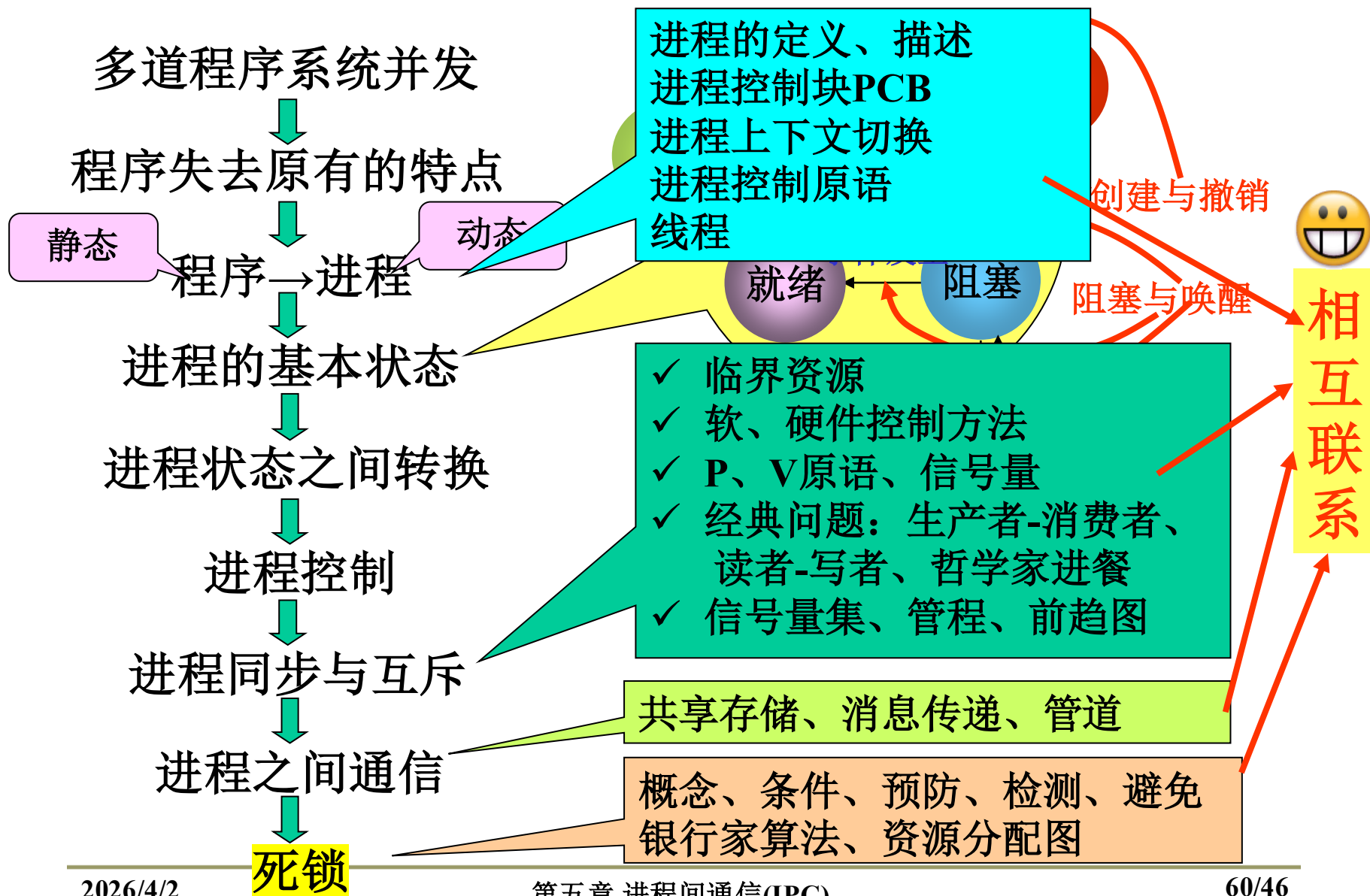


进程的基本状态





小结





继续学习第6章

死锁

